Rival Payments — Transaction Ingestion API

Integration guide for the Whish Money technical team

This document describes the two HTTP APIs Whish needs to push transactions into the Rival Payments system:

1. `GET /ingest/companies` — discover which operating entities you can post transactions for.
2. `POST /ingest/whish/transactions` — submit a transaction for an operating entity.

The normal flow is: call **(1)** once on start-up (and refresh periodically) to learn the list of entities and their IDs, then call **(2)** for every transaction, using the `operatingId` returned by (1).

---

1. Connection basics

Item	Value
Base URL	`https://portal.rivalpayments.com/v1`
Protocol	HTTPS only
Content type	`application/json` (request and response)
Authentication	`Authorization: Bearer <API_KEY>` header on every request
Character encoding	UTF-8

Authentication

Every request must carry your API key in the `Authorization` header. Your key is:

```
Authorization: Bearer
tsk_GXHNAMY9_6de89005dee6a2a1a7f5fb720468c0cf4194b6ba2804d51556f5d5e0ee63f9dd
```

- This key identifies you as the `whish` source. All requests use it unchanged.
- The key is secret — treat it like a password, send it only over HTTPS, and never embed it in client-side code or logs.
- A missing or malformed header returns **401 Unauthorized**; an invalid or revoked key also returns **401**.

Rate limit

Requests are rate-limited **per API key**:

- **300 requests per rolling 60-second window** (≈ 5 requests/second, burst-friendly).
- Exceeding the limit returns **429** with code `RATE_LIMITED`. Back off and retry.

Response envelope

Every JSON response uses the **same envelope**, so you can branch on one field and read the status code straight from the body:

- **Success:** `{ "success": true, "statusCode": <code>, "data": <payload> }` — your result is always under `data`.
- **Error:** `{ "success": false, "statusCode": <code>, "error": { "code", "message", "details" } }`.

`statusCode` mirrors the HTTP status line. All the examples below show the full envelope.

---

2. API 1 — Get companies (discovery)

Returns every operating entity that is currently **active and visible** to sources. Use this to learn the `operatingId` values you will post transactions against, plus their display names and default currency.

Request

```
GET /v1/ingest/companies
Authorization: Bearer <API_KEY>
```

No query parameters and no request body.

Response — `200 OK`


```
{
  "success": true,
  "statusCode": 200,
  "data": {
    "data": [
      {
        "operatingId": "3f8a1c20-9b2e-4d77-8a1f-7c2e5b0d4a11",
        "slug": "acme-retail",
        "name": "ACME Retail",
        "defaultCurrency": "USD"
      },
      {
        "operatingId": "b14d77e2-1f53-49c0-9e8a-2d6c0f3b9a44",
        "slug": "globex-fuel",
        "name": "Globex Fuel",
        "defaultCurrency": "USD"
      }
    ]
  }
}
```


The envelope's `data` holds the list payload `{ "data": [...] }`, so the array of entities is at **`data.data`**.

Field reference

Field	Type	Description
`data.data`	array	List of available operating entities. May be empty if none are currently visible.
`data.data[].operatingId`	string (UUID)	**The value you must send as `operatingId` when submitting a transaction.** Stable — safe to cache.
`data.data[].slug`	string	Human-friendly short identifier. For display only.
`data.data[].name`	string	Entity display name. For display only.
`data.data[].defaultCurrency`	string	The entity's default settlement currency (currently `USD`).

> **Note:** Only entities that Rival Payments has marked active and visible appear here. One that is hidden or inactive will not appear and cannot receive transactions. Refresh this list periodically (e.g. every few minutes) so newly onboarded entities become available.

---

3. API 2 — Ingest a transaction

Submits a single completed transaction for an operating entity. On success the transaction is recorded immediately with status `COMPLETED` and the entity's balance is credited (net of any commission).

Request

```
POST https://portal.rivalpayments.com/v1/ingest/whish/transactions
Authorization: Bearer <API_KEY>
Content-Type: application/json
```

- Use this exact URL — `whish` is your fixed source segment. Posting to any other source path is rejected with **403 Forbidden**, since your API key is bound to `whish`.

Request body

```
{
  "operatingId": "3f8a1c20-9b2e-4d77-8a1f-7c2e5b0d4a11",
  "externalTransactionId": "WISH-2026-000123456",
  "amount": "150.00",
  "currency": "USD",
  "payload": {
    "customer": "John Doe",
    "tradingAccount": "0096170123456",
    "phone": "0096170123456",
    "note": "Top-up via Whish"
  }
}
```

Body field reference

Field	Type	Required	Rules
`operatingId`	string (UUID)	**Yes**	An `operatingId` returned by `GET /ingest/companies`. Must be active and visible.
`externalTransactionId`	string	**Yes**	Your own unique reference for this transaction. 1–200 characters. Used for de-duplication (see Idempotency).
`amount`	string (decimal)	**Yes**	The transaction amount as a **decimal string** (never a number/float). Must be positive. Up to 8 decimal places and up to 12 integer digits. Examples: `"150.00"`, `"49.995"`.
`currency`	string	**Yes**	Currency code. Currently only `USD` is supported.
`payload`	object	No (defaults to `{}`)	Descriptive details about the transaction. See below.
`idempotencyKey`	string	No	**Rarely needed.** Optional explicit idempotency key (8–120 characters). You can ignore it: `externalTransactionId` already de-duplicates your transactions (see Idempotency).

`payload` **object** — all sub-fields are optional. The following keys are recognised; **any additional keys you send are also stored verbatim** for audit:

`payload` key	Type	Max length	Meaning
`customer`	string	200	The customer who paid.
`tradingAccount`	string	200	The trading account they paid (account/number/etc.).
`phone`	string	40	A contact phone number.
`note`	string	1000	Free-text note.
(any extra keys)	any	—	Preserved as-is for audit.

Response — `201 Created`


```
{
  "success": true,
  "statusCode": 201,
  "data": {
    "id": "a7c3e9f1-2b44-4d8e-9c10-5f6a7b8c9d0e",
    "operatingId": "3f8a1c20-9b2e-4d77-8a1f-7c2e5b0d4a11",
    "sourceId": "5d2b1a90-7e34-4c11-b8f2-1a9c3e7d6b50",
    "sourceCode": "whish",
    "externalTransactionId": "WISH-2026-000123456",
    "amount": "150.00",
    "currency": "USD",
    "status": "COMPLETED",
    "customer": "John Doe",
    "tradingAccount": "0096170123456",
    "phone": "0096170123456",
    "note": "Top-up via Whish",
    "processedAt": "2026-06-04T12:34:56.000Z",
    "createdAt": "2026-06-04T12:34:56.000Z"
  }
}
```


The transaction object is under **`data`** (fields below are relative to `data`).

Response field reference

Field	Type	Description
`id`	string (UUID)	Rival Payments' internal transaction ID.
`operatingId`	string (UUID)	The operating entity credited.
`sourceId`	string (UUID)	The source (you) that submitted it.
`sourceCode`	string	Always `whish` (your source).
`externalTransactionId`	string	Echo of the reference you sent.
`amount`	string (decimal)	The amount you submitted.
`currency`	string	Currency code.
`status`	string	`COMPLETED` for a newly accepted transaction.
`customer` / `tradingAccount` / `phone` / `note`	string | null	Echo of the descriptive payload fields (null if not provided).
`processedAt`	string (ISO 8601) | null	When the transaction was processed.
`createdAt`	string (ISO 8601)	When the record was created.

Idempotency (avoid double-posting)

The endpoint is safe to retry. A transaction is treated as a **duplicate** (and the original record is returned instead of creating a new one) when:

1. The pair **(your source, `externalTransactionId`)** has already been seen — this is automatic and is all you need, **or**
2. *(advanced, optional)* you send an `idempotencyKey` that was already used for this entity.

In both cases you receive the **existing** transaction in the response and **no new money is booked**. This means: if you get a network timeout, you can safely resend the exact same request — you will not double-credit the entity.

> Reusing the *same* `externalTransactionId` for a *different* entity is rejected as a conflict (`409`). Keep your `externalTransactionId` globally unique per transaction.

---

4. Error handling

All errors use the same envelope as success responses, with the failure under `error`:

```
{
  "success": false,
  "statusCode": 400,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable description",
    "details": {}
  }
}
```

`details` is optional and, for validation errors, lists the specific fields that failed. You can detect failure by `success === false` (or the presence of `error`); `statusCode` echoes the HTTP status.

Status codes

HTTP	`code`	When
`200` / `201`	—	Success (200 for GET companies, 201 for transaction created).
`400`	`VALIDATION_ERROR`	Body failed validation (missing/invalid `operatingId`, bad `amount`, unsupported `currency`, etc.).
`401`	`UNAUTHORIZED`	Missing, malformed, invalid, or revoked API key.
`403`	`FORBIDDEN`	The URL used a source other than `whish`, the entity is hidden/not available to sources, or the request was made without your source API key.
`400`	`VALIDATION_ERROR`	Target entity or source not found / inactive (returned as a validation error).
`409`	`CONFLICT` / `WRITE_CONFLICT`	Constraint conflict (e.g. same `externalTransactionId` reused across companies) or a transient write conflict — safe to retry.
`429`	`RATE_LIMITED`	Rate limit exceeded; back off and retry.
`500`	`INTERNAL_ERROR`	Unexpected server error. Retry with backoff; contact Rival Payments if persistent.

Recommended retry policy

- **Network errors / timeouts / `429` / `409 WRITE_CONFLICT` / `5xx`:** retry with exponential backoff. Because of idempotency, resending the identical request is safe.
- **`400` / `401` / `403`:** do **not** blindly retry — the request itself is wrong; fix the payload/credentials/configuration first.

---

5. End-to-end example (cURL)

Step 1 — discover companies

```
curl -sS https://portal.rivalpayments.com/v1/ingest/companies \
  -H "Authorization: Bearer
tsk_GXHNAMY9_6de89005dee6a2a1a7f5fb720468c0cf4194b6ba2804d51556f5d5e0ee63f9dd"
```

Step 2 — submit a transaction (using an `operatingId` from step 1)

```
curl -sS -X POST \
  https://portal.rivalpayments.com/v1/ingest/whish/transactions \
  -H "Authorization: Bearer
tsk_GXHNAMY9_6de89005dee6a2a1a7f5fb720468c0cf4194b6ba2804d51556f5d5e0ee63f9dd" \
  -H "Content-Type: application/json" \
  -d '{
    "operatingId": "3f8a1c20-9b2e-4d77-8a1f-7c2e5b0d4a11",
    "externalTransactionId": "WISH-2026-000123456",
    "amount": "150.00",
    "currency": "USD",
    "payload": {
      "customer": "John Doe",
      "tradingAccount": "0096170123456",
      "phone": "0096170123456",
      "note": "Top-up via Whish"
    }
  }'
```

---

6. Integration summary

Everything you need is already fixed:

1. **API key:** `tsk_GXHNAMY9_6de89005dee6a2a1a7f5fb720468c0cf4194b6ba2804d51556f5d5e0ee63f9dd`
2. **Transaction URL:** `https://portal.rivalpayments.com/v1/ingest/whish/transactions`
3. **Discovery URL:** `https://portal.rivalpayments.com/v1/ingest/companies`

The `operatingId` values are **not** provided manually — you obtain them yourself from the discovery endpoint above.

Document version 1.0 — for the Whish Money technical team.